TECHNICAL GUIDE FOR TEAM

Cross-Repository Integration & Live Bridge Guide

How AnyLogic (.alp), AI-3D-Robotics-Lab (Three.js/Node), and mdc-robotics-simulation-twin (Python/Streamlit) Communicate in Real-Time

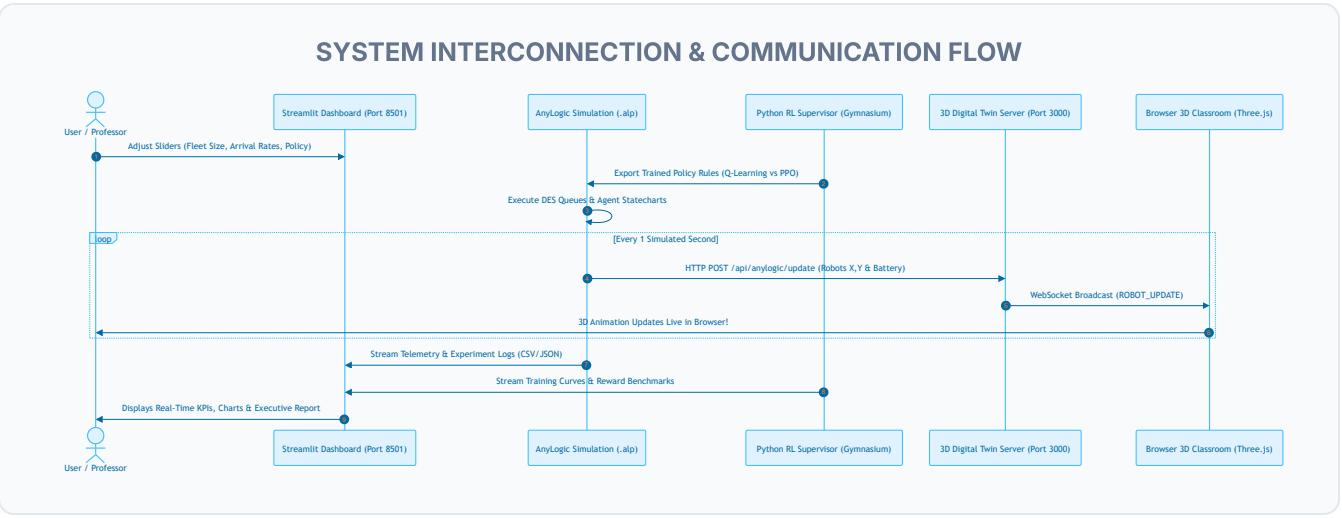
PRIMARY ENGINE	3D DIGITAL TWIN	AI & DASHBOARD
AnyLogic 8 (Java)	Node.js Express + Three.js (Port 3000)	Python Gym + Streamlit (Port 8501)

TEAM

Jorge, Jordan, Illia, Ricardo

1. The Big Picture: Three Systems, One Unified Ecosystem

Our group project seamlessly connects three specialized platforms so that each team member's component communicates with the others without friction:



2. How AnyLogic Talks to the 3D Digital Twin (Channel 1)

> **Key Fact:** The backend receiver is ALREADY BUILT in our `AI-3D-Robotics-Lab` repository!

Inside `AI-3D-Robotics-Lab/digital_twin_app/server.js` (lines 58–66), we already have the dedicated endpoint:

```
app.post('/api/anylogic/update', (req, res) => {
  const { robots, source } = req.body;
  if (Array.isArray(robots)) {
    world.mergeRobots(robots);
    broadcast('ROBOT_UPDATE', { robots: world.getRobots(), source: source || 'AnyLogic
  }
  res.json({ success: true, count: world.getRobots().length });
});
```

The AnyLogic Transmitter Code (Java):

In AnyLogic, inside a periodic **Event** block (set to repeat every 1.0 simulated second), Jorge drops this native Java code:

```
// Native Java inside AnyLogic Model
String json = String.format(
    "{\"source\":\"AnyLogic\",\"robots\":[{\"id\":\"amr01\",\"x\":%.2f,\"y\":%.2f,\"st
    rover1.getX(), rover1.getY(), rover1.state, (int)(rover1.batteryLevel * 100)
);

java.net.http.HttpClient client = java.net.http.HttpClient.newHttpClient();
java.net.http.HttpRequest request = java.net.http.HttpRequest.newBuilder()
    .uri(java.net.URI.create("http://localhost:3000/api/anylogic/update"))
    .header("Content-Type", "application/json")
    .POST(java.net.http.HttpRequest.BodyPublishers.ofString(json))
    .build();

client.sendAsync(request, java.net.http.HttpResponse.BodyHandlers.discarding());
```

Result: When AnyLogic runs, it transmits coordinates asynchronously over HTTP. The Node server immediately broadcasts them over WebSockets to the Three.js 3D canvas, causing the 3D rovers in the web browser to mirror AnyLogic's movements!

3. How Reinforcement Learning Connects to AnyLogic & UI (Channel 2)

Jordan develops and trains the RL algorithms in Python inside `rl_models/`:

- **The Training Cycle:** `train_qlearning.py` and `train_ppo.py` interact with the custom environment `robotics_lab_env.py` to learn optimal dispatching, parcel batching, and opportunistic charging thresholds.
- **Exporting to AnyLogic:** The learned decision rules (e.g. state-action Q-table and PPO policy lookup) export as `q_table.npy` and JSON policy logic. AnyLogic reads these rules in Java during simulation runs to replace naive FIFO dispatching.
- **Exporting to Streamlit:** The training metrics automatically write to `q_learning_results.json` and `ppo_results.json`.

4. How AnyLogic & RL Feed the Streamlit Dashboard (Channel 3)

Illia's Streamlit dashboard (`dashboard/app.py`) acts as the executive control room:

```
# Inside dashboard/app.py
# Loads Jordan's RL training outputs:
with open("../rl_models/q_learning_results.json") as f:
    q_data = json.load(f)
with open("../rl_models/ppo_results.json") as f:
    ppo_data = json.load(f)

# Renders dynamic Plotly comparison curves and KPI cards
st.plotly_chart(render_comparison(q_data, ppo_data))
```

When Ricardo executes the 5 AnyLogic experiments (Optimization, Monte Carlo, Sensitivity Analysis), the resulting data tables populate Tab 3 of the dashboard.

5. Step-by-Step 3-Terminal Launch & Testing Sequence

To run and demonstrate the entire ecosystem live, open three terminal windows:

> **Terminal 1: Start the 3D Web Digital Twin Server (Port 3000)**
>
> ```
> cd "D:\Miami Dade College\AI-3D-Robotics-Lab\digital_twin_app"
> npm start
> # Server listening at: http://localhost:3000
> ```

Terminal 2: Launch the Streamlit Analytical Dashboard (Port 8501)

```
cd "D:\Miami Dade College\Simulation For AI\mdc_robotics_simulation_twin\dashboard
streamlit run app.py
# Dashboard listening at: http://localhost:8501
```

Step 3: Run the AnyLogic Model (.alp)

Open `MDC_Robotics_Lab_Simulation.alp` in AnyLogic 8 and click **Run**. AnyLogic executes the DES queues, streams (X,Y) positions to Port 3000, and updates the metrics on Port 8501!

6. Summary of Team Responsibilities

Team Member	Primary Tech Stack	What They Build & Deliver
Jorge A Mendoza	AnyLogic 8 (Java)	Builds the multi-method simulation model and sets up the HTTP live bridge to Port 3000.
Jordan Gonzalez	Python (Gymnasium, SB3)	Builds the RL training loops and exports `q_learning_results.json` and `ppo_results.json`.
Illia Shybanov	Python (Streamlit, Plotly)	Builds the 5-tab web dashboard and connects inputs/sliders to the simulation metrics.
Ricardo Molina Gonzalez	AnyLogic Experiments & Docs	Runs the 5 AnyLogic experiments, compiles data, and drafts the technical report and slides.